contents

Front matter

foreword

preface

acknowledgements

about this book

about the authors

about the cover illustration

Part 1. **From 8 to 11 and beyond!**

1 **Introducing modern Java**

1.1 The language and the platform

1.2 The new Java release model

1.3 Enhanced type inference (var keyword)

1.4 Changing the language and the platform

Sprinkling some sugar

Changing the language

JSRs and JEPs

Incubating and preview features

1.5 Small changes in Java 11

Collections factories (JEP 213)

Remove enterprise modules (JEP 320)

HTTP/2 (Java 11)

Single-file source-code programs (JEP 330)

2 Java modules

2.1 Setting the scene

Project Jigsaw

The module graph

Protecting the internals

New access control semantics

2.2 Basic modules syntax

Exporting and requiring

Transitivity

2.3 Loading modules

Platform modules

Application modules

Automatic modules

Unnamed module

2.4 Building a first modular app

Command-line switches for modules

Executing a modular app

Modules and reflection

2.5 Architecting for modules

Split packages

Java 8 Compact Profiles

Multi-release JARs

2.6 Beyond modules

3 Java 17

3.1 Text Blocks

3.2 Switch Expressions

3.3 Records

Nominal typing

Compact record constructors

3.4 Sealed Types

3.5 New form of instanceof

3.6 Pattern Matching and preview features

Part 2. Under the hood

4 Class files and bytecode

4.1 Class loading and class objects

Loading and linking

Class objects

4.2 Class loaders

Custom class loading

Modules and class loading

4.3 Examining class files

Introducing javap

Internal form for method signatures

The constant pool

4.4 Bytecode

Disassembling a class

The runtime environment

Introduction to opcodes

Load and store opcodes

Arithmetic opcodes

Execution flow control opcodes

Invocation opcodes

Platform operation opcodes

Shortcut opcode forms

4.5 Reflection

Introducing reflection

Combining class loading and reflection

Problems with reflection

5 Java concurrency fundamentals

5.1 Concurrency theory primer

But I already know about Thread

Hardware

Amdahl's law

Explaining Java's threading model

Lessons learned

5.2 Design concepts

Safety and concurrent type safety

Liveness

Performance

Reusability

How and why do the forces conflict?

Sources of overhead

5.3 Block-structured concurrency (pre-Java 5)

Synchronization and locks

The state model for a thread

Fully synchronized objects

Deadlocks

Why synchronized?

The volatile keyword

Thread states and methods

Immutability

5.4 The Java Memory Model (JMM)

5.5 Understanding concurrency through bytecode

Lost Update

Synchronization in bytecode

Synchronized methods

Unsynchronized reads

Deadlock revisited

Deadlock resolved, revisited

Volatile access

6 JDK concurrency libraries

6.1 Building blocks for modern concurrent applications

6.2 Atomic classes

6.3 Lock classes

Condition objects

6.4 CountDownLatch

6.5 ConcurrentHashMap

Understanding a simplified HashMap

Limitations of Dictionary

Approaches to a concurrent Dictionary

Using ConcurrentHashMap

6.6 CopyOnWriteArrayList

6.7 Blocking queues

Using BlockingQueue APIs

Using WorkUnit

6.8 Futures

CompletableFuture

6.9 Tasks and execution

Modeling tasks

Executors

Single-threaded executor

Fixed-thread pool

Cached thread pool

ScheduledThreadPoolExecutor

7 **Understanding Java performance**

7.1 Performance terminology: Some basic definitions

Latency

Throughput

Utilization

Efficiency

Capacity

Scalability

Degradation

7.2 A pragmatic approach to performance analysis

Know what you're measuring

Know how to take measurements

Know what your performance goals are

Know when to stop

Know the cost of achieving higher performance

Know the dangers of premature optimization

7.3 What went wrong? Why do we have to care?

Moore's law

Understanding the memory latency hierarchy

7.4 Why is Java performance tuning hard?

The role of time in performance tuning

Understanding cache misses

7.5 Garbage collection

Basics

Mark and sweep

Areas of memory

Young collections

Full collections

Safepoints

G1: Java's default collector

The Parallel collector

GC configuration parameters

7.6 JIT compilation with HotSpot

Why have dynamic compilation?

Introduction to HotSpot

Inlining methods

Dynamic compilation and monomorphic calls

Reading the compilation logs

Deoptimization

7.7 JDK Flight Recorder

Flight Recorder

Mission Control

Part 3. Non-Java languages on the JVM

8 Alternative JVM languages

8.1 Language zoology

Interpreted vs. compiled languages

Dynamic vs. static typing

Imperative vs. functional languages

Reimplementation vs. original

8.2 Polyglot programming on the JVM

Why use a non-Java language?

Up-and-coming languages

Languages we could have picked but didn't

8.3 How to choose a non-Java language for your project

Is the project area low-risk?

Does the language interoperate well with Java?

Is there good tooling and test support for the language?

How hard is the language to learn?

Are there lots of developers using this language?

8.4 How the JVM supports alternative languages

Performance

Runtime environments for non-Java languages

Compiler fictions

9 Kotlin

9.1 Why use Kotlin?

Installing

9.2 Convenience and conciseness

Starting with less

Variables

Equality

Functions

Collections

Express yourself

9.3 A different view of classes and objects

Data classes

9.4 Safety

Null safety

Smart casting

9.5 Concurrency

9.6 Java interoperability

10 Clojure: A different view of programming

10.1 Introducing Clojure

Hello World in Clojure

Getting started with the REPL

Making a mistake

Learning to love the brackets

10.2 Looking for Clojure: Syntax and semantics

Special forms bootcamp

Lists, vectors, maps, and sets

Arithmetic, equality, and other operations

Working with functions in Clojure

Loops in Clojure

Reader macros and dispatch

10.3 Functional programming and closures

10.4 Introducing Clojure sequences

Sequences and variable-arity functions

10.5 Interoperating between Clojure and Java

Calling Java from Clojure

The nature of Clojure calls

The Java type of Clojure values

Using Clojure proxies

Exploratory programming with the REPL

Using Clojure from Java

10.6 **Macros**

Part 4. **Build and deployment**

11 **Building with Gradle and Maven**

11.1 Why build tools matter for a well-grounded developer

Automating tedious operations

Managing dependencies

Ensuring consistency between developers

11.2 Maven

The build lifecycle

Commands/POM intro

Building

Controlling the manifest

Adding another language

Testing

Dependency management

Reviewing

Moving beyond Java 8

Multirelease JARs in Maven

Maven and modules

Authoring Maven plugins

11.3 Gradle

Installing Gradle

Tasks

What's in a script?

Using plugins

Building

Work avoidance

Dependencies in Gradle

Adding Kotlin

Testing

Automating static analysis

Moving beyond Java

Using Gradle with modules

Customizing

12 **Running Java in containers**

12.1 Why containers matter for a well-grounded developer

Host operating systems vs. virtual machines vs. containers

Benefits of containers

Drawbacks of containers

12.2 Docker fundamentals

Building Docker images

Running Docker containers

12.3 Developing Java applications with Docker

Selecting your base image

Building an image with Gradle

Running the build in Docker

Ports and hosts

Local development with Docker Compose

Debugging in Docker

Logging with Docker

12.4 Kubernetes

12.5 Observability and performance

Observability

Performance in containers

13 Testing fundamentals

13.1 Why we test

13.2 How we test

13.3 Test-driven development

TDD in a nutshell

A TDD example with a single use case

13.4 Test doubles

Dummy object

Stub object

Fake object

Mock object

Problems with mocking

13.5 From JUnit 4 to 5

14 Testing beyond JUnit

14.1 Integration testing with Testcontainers

Installing testcontainers

An example with Redis

Gathering container logs

An example with Postgres

An example for end-to-end testing with Selenium

14.2 Specification-style testing with Spek and Kotlin

14.3 Property-based testing with Clojure

clojure.test

clojure.spec

test.check

clojure.spec and test.check

Part 5. Java frontiers

15 Advanced functional programming

15.1 Introduction to functional programming concepts

Pure functions

Immutability

Higher-order functions

Recursion

Closures

Laziness

Currying and partial application

15.2 Limitations of Java as a FP language

Pure functions

Mutability

Higher-order functions

Recursion

Closures

Laziness

Currying and partial application

Java's type system and collections

15.3 Kotlin FP

Pure and higher-order functions

Closures

Currying and partial application

Immutability

Tail recursion

Lazy evaluation

Sequences

15.4 **Clojure FP**

Comprehensions

Lazy sequences

Currying in Clojure

16 **Advanced concurrent programming**

16.1 The Fork/Join framework

A simple F/J example

Parallelizing problems for F/J

Work-stealing algorithms

16.2 Concurrency and functional programming

CompletableFuture revisited

Parallel streams

16.3 Under the hood with Kotlin coroutines

How coroutines work

Coroutine scoping and dispatching

16.4 Concurrent Clojure

Persistent data structures

Futures and pcalls

Software transactional memory

Agents

17 Modern internals

17.1 Introducing JVM internals: Method invocation

Invoking virtual methods

Invoking interface methods

Invoking “special” methods

Final methods

17.2 Reflection internals

17.3 Method handles

MethodHandle

MethodType

Looking up method handles

Reflection vs. proxies vs. method handles

17.4 Invokedynamic

Implementing lambda expressions

17.5 Small internal changes

String concatenation

Compact strings

Nestmates

17.6 Unsafe

17.7 Replacing Unsafe with supported APIs

VarHandles

Hidden classes

18 Future Java

18.1 Project Amber

18.2 Project Panama

Foreign Function and Memory API

18.3 Project Loom

Virtual threads

Thread builders

Programming with virtual threads

When will Project Loom arrive?

18.4 Project Valhalla

Changing the language model

Consequences of value objects

Generics revisited

18.5 Java 18

Appendix A. Selecting your Java

Appendix B. Recap of streams in Java 8

index